KTÜ OF TEKNOLOJI FAKÜLTESI · YAZILIM MÜHENDISLIĞI BÖLÜMÜ

Nesne Yönelimli Programlama

2026–2027 Güz Dönemi · Öğrenci Ders Notu

HAFTA 04

Kapsülleme ve Erişim Belirleyicileri

private · varsayılan · protected · public — veriyi kim, nereden, hangi kuralla değiştirebilir?

ÖZET

Hafta 3'te nesne yöneliminin dört ilkesini kuşbakışı gördük ve kapsüllemenin bu dördünün **taşıyıcı sütunu** olduğunu söyledik. Bu hafta o sütunu tek tek söküp inceliyoruz. Java'nın dört erişim düzeyini (`private`, varsayılan, `protected`, `public`) tanıyacak; getter/setter düzeninin ne olduğunu ve ne **olmadığını** ayırt edeceğiz. Doğrulama mantığını setter'a taşımayı, salt-okunur ve `final` alan tasarlamayı, değişmez (immutable) sınıf yazmayı ve paket kavramının erişimle ilişkisini çalışan kod üzerinde göreceğiz.

Ders	Nesne Yönelimli Programlama (Zorunlu)
Program / Sınıf	Yazılım Mühendisliği · 2. Sınıf · Güz
Haftalık Yük	2 saat teori + 2 saat laboratuvar (toplam 4 saat)
Öğretim Elemanı	Dr. Öğr. Üyesi Rıfat BENVENİSTE
Laboratuvar Sorumlusu	Arş. Gör. Betül MUMCU
Programlama Dili	Java (JDK 21 LTS)
Geliştirme Ortamı	Visual Studio Code + Extension Pack for Java

İçindekiler

1. Bu Haftanın Yol Haritası ... 3

1.1. Ön Koşul Bilgi Kontrolü ... 3

1.2. Haftanın Zaman Planı ... 4

2. Motivasyon: Açık Bırakılan Alanın Bedeli ... 5

2.1. Sorunu Üreten Kod ... 5

2.2. Aynı İstekler, Kapsüllenmiş Sınıfta ... 6

2.3. Sorunun Asıl Boyutu: Bakım ... 7

3. Kapsülleme: Tanım ve Sözdizimi ... 9

3.1. En Küçük Kapsülleme: Alanı Kapat, Kapıyı Aç ... 9

3.2. Getter: Okuma Kapısı ... 10

3.3. Setter: Denetimli Yazma Kapısı ... 10

3.4. Türetilmiş Getter: Alanı Olmayan Kapı ... 11

3.5. Sınıfın Tam Listesi ... 12

4. Derinleşme ... 13

4.1. Dört Erişim Düzeyi ... 13

4.2. Paket Kavramı ve Erişim Sınırı ... 14

Aynı paketten bakış ... 14

Farklı paketten bakış ... 15

4.3. `final` Alan ve Salt-Okunur Tasarım ... 16

4.4. Değişmez (Immutable) Sınıf ... 17

4.5. Setter mi, Anlamlı Metot mu? ... 19

4.6. Aynı Bilgiyi İki Alanda Tutmamak ... 21

5. C'den Java'ya Köprü ... 23

6. Sık Yapılan Hatalar ... 24

6.1. Düşünmeden Yazılan Getter/Setter Çifti ... 24

6.2. Aynı Sınıfın İçinden `private`'a Erişilemeyeceğini Sanmak ... 24

6.3. `final` ile Değişmezliği Karıştırmak ... 24

6.4. Getter'ın Değiştirilebilir Nesne Döndürmesi ... 25

6.5. Yapıcıda Doğrulamayı Atlamak ... 25

6.6. Derleyici İletileri Sözlüğü ... 25

7. İyi Tasarım İlkeleri ... 26

7.1. Adlandırma Kuralları ... 26

8. Bir Sonraki Haftaya Köprü ... 27

9. Haftanın Özeti ... 27

10. Alıştırmalar ... 28

10.1. Kavram Soruları ... 28

10.2. Kod Okuma ve Hata Bulma ... 28

10.3. Programlama Alıştırmaları ... 29

11. Öz Değerlendirme 31

Öz Değerlendirme Kontrol Listesi 31

12. Kaynaklar 31

12.1. Ders Kitapları 31

12.2. Çevrimiçi Kaynaklar 32

12.3. İletişim ve Destek 32

1. Bu Haftanın Yol Haritası

İlk üç hafta boyunca sınıflarımızın alanlarını bilinçli bir sadeleştirme uğruna `public` bıraktık. Bu, öğrenmenin başında doğru bir karardı: dikkatimizi sınıf–nesne ayrımına ve yapıcıya vermek istiyorduk. Ancak bu kararın bir bedeli vardı ve bu hafta o bedeli ödemeyi bırakıyoruz.

Bedel şudur: alanı `public` olan bir sınıf **kendi verisinin doğruluğundan sorumlu olamaz**. Bir öğrencinin yaşı eksi üç olabilir, not ortalaması 350 olabilir, adı boş dize olabilir. Derleyici bunların hiçbirine itiraz etmez; çünkü dil açısından ortada bir hata yoktur. Hata **tasarımdadır**.

Kapsülleme (encapsulation), bu sorumluluğu sınıfın kendisine geri veren ilkedir. İki bileşeni vardır ve ikisi birlikte anlam kazanır: (i) veriyi dışarıdan **gizlemek**, (ii) veriyi değiştiren **kuralı** sınıfın içine taşımak. Bu haftanın en sık yapılan yanlışı, birincisini yapıp ikincisini atlamaktır — yani her alana düşünmeden bir getter ve bir setter yazmaktır.

◎ Bu Bölümün Sonunda Öğrenci

- Kapsüllemenin yalnızca “alanları `private` yapmak” olmadığını; veriyi değiştiren **iş kuralını** da sınıfın içine taşımak olduğunu örnekle açıklayabilir.
- Java'nın dört erişim düzeyini (`private`, varsayılan/paket, `protected`, `public`) **görünürlük matrisi** üzerinden karşılaştırabilir ve doğru düzeyi seçebilir.
- Bir alan için getter ve/veya setter yazılıp yazılmayacağına **gerekçeli** karar verebilir; refleks olarak ikisini birden yazmanın neden kapsülleme sayılmadığını açıklayabilir.
- Doğrulama mantığını setter'ın içine taşıyabilir; geçersiz değer geldiğinde nesnenin **eski geçerli durumunu koruduğunu** gösterebilir.
- `final` alanla salt-okunur alan tasarlayabilir; `final`'ın “bağı” dondurduğunu ama nesne “içeriğini” dondurmadığını ayırt edebilir.
- Değişmez (immutable) bir sınıfı dört kuralına uygun biçimde yazabilir ve değişmezliğin paylaşımı neden güvenli kıldığını açıklayabilir.
- Paket kavramını ve `protected` ile varsayılan erişimin paket sınırındaki davranışını çalışan kod üzerinde gösterebilir.
- Bir alanın karşılığı olan **türetilmiş getter** yazabilir ve aynı bilgiyi iki alanda tutmanın neden tutarsızlık ürettiğini gerekçelendirebilir.

1.1. Ön Koşul Bilgi Kontrolü

Aşağıdaki soruların tamamını Hafta 1–3 bilgisiyle yanıtlayabiliyor olmalısınız. Yanıtlayamadığınız madde varsa laboratuvar öncesinde ilgili bölümü tekrar etmeniz önerilir.

1. Bir sınıfın **alanı** ile **yerel değişkeni** hangi noktalarda ayrılır? Hangisi nesneyle birlikte yaşar?
2. `this.ad = ad;` satırında `this` neyi işaret eder ve neden gereklidir?
3. Yapıcı metodun görevi nedir? Yapıcı çalışmadan nesne kullanılabilir mi?
4. Hafta 3'te kapsüllemeyi hangi cümleyle tanımlamıştık? `BankaHesabi` örneğinde `bakiye` alanına neden doğrudan erişilemiyordu?

5. `toString()` metodunu ezdiğinizde `System.out.println(nesne)` çağrısı ne yapar?

1.2. Haftanın Zaman Planı

Oturum	Süre	İçerik	Çıktı
Teori 1	45 dk	Motivasyon problemi · kapsüllemenin tanımı · `private` ve getter/setter düzeni	Açık alanın bedelini örnekle gösterebilme
Teori 2	45 dk	Dört erişim düzeyi · paket kavramı · `final` ve salt-okunur alan · değişmez sınıf	Doğru erişim düzeyini gerekçeli seçebilme
Lab 1	60 dk	Görev 0–2: ortam kontrolü · kapsülleme ihlalini düzeltme · doğrulamalı setter	Çalışan ve doğrulayan iki sınıf
Lab 2	60 dk	Görev 3–5: salt-okunur alan · değişmez sınıf · (bonus) paket düzeni	Teslim edilecek `.zip` paketi

Tablo 4.1 — Hafta 4'ün oturum planı ve beklenen çıktılar

> **ⓘ BİLGİ**
>
> Bu hafta yeni bir sözdizimi bolluğu yoktur; öğrenilecek anahtar sözcük sayısı azdır (`private`, `protected`, `public`, `final`, `package`). Zorluk sözdiziminde değil **karar vermededir**: hangi alana hangi kapıyı açacağınıza karar vermek, bu haftanın asıl becerisidir.

2. Motivasyon: Açık Bırakılan Alanın Bedeli

Hafta 1'de yazdığımız `Ogrenci` sınıfının alanları `public` idi. Bu sınıfın sadeleştirilmiş bir kopyasını alalım ve gerçek bir projede olabileceği gibi **farklı modüllerin** ona dokunduğunu düşünelim: kayıt modülü, sınav modülü, arayüz modülü.

2.1. Sorunu Üreten Kod

KapsullemeYok.java

```java
/* Ayni dosyadaki paket-ozel yardimci sinif (public degil). */
class AcikOgrenci {
    public String ad;
    public int yas;
    public double notOrtalamasi;

    public void yaz() {
        System.out.printf("ad='%s' | yas=%d | ortalama=%.1f%n", ad, yas, notOrtalamasi);
    }
}
```

Kod 4.1 — Alanları açık bırakılmış sınıf

Bu sınıfta üç alan da `public`. Sınıfın bu alanların anlamlı bir değere sahip olmasını sağlayacak **hiçbir aracı** yoktur; çünkü değeri atayan kod sınıfın dışındadır.

KapsullemeYok.java

```java
    public static void main(String[] args) {
        AcikOgrenci o = new AcikOgrenci();
        o.ad = "Elif Kaya";
        o.yas = 20;
        o.notOrtalamasi = 78.5;
        o.yaz();

        System.out.println();
        System.out.println("--- Farkli modullerden gelen atamalar ---");

        // (a) Kayit modulu: yasi yanlislikla eksi yaziyor
        o.yas = -3;
        // (b) Sinav modulu: 100 uzeri not yaziyor
        o.notOrtalamasi = 350.0;
        // (c) Arayuz modulu: adi bosaltiyor
        o.ad = "";

        o.yaz();

        System.out.println();
        System.out.println("Nesne gecersiz durumda; derleyici hic sikayet etmedi.");
        System.out.println("Hatanin kaynagi bu satirlar degil, alanlarin acik olmasidir.");
    }
```

Kod 4.2 — Üç farklı modülün aynı nesneye dokunması

```
▸ TERMINAL — java KapsullemeYok
```

```
ad='Elif Kaya' | yas=20 | ortalama=78.5

--- Farkli modullerden gelen atamalar ---
ad='' | yas=-3 | ortalama=350.0

Nesne gecersiz durumda; derleyici hic sikayet etmedi.
Hatanin kaynagi bu satirlar degil, alanlarin acik olmasidir.
```

Çıktının ikinci satırına dikkat ediniz: `ad=''`, `yas=-3`, `ortalama=350.0`. Ortada **geçersiz bir nesne** vardır ve bu nesne programın geri kalanında dolaşmaya devam edecektir. Rapor modülü bu öğrenciyi listeleyecek, ortalama hesabı bozulacak, hata bir noktada patlak verdiğinde ise **hatanın kaynağı çoktan kaybolmuş** olacaktır.

> ✕ **SIK YAPILAN HATA**
>
> Bu tasarımda hatanın oluştuğu yer ile hatanın **fark edildiği** yer birbirinden uzaktır. `o.yas = -3;` satırı sessizce çalışır; sorun haftalar sonra bir raporda ortaya çıkar. Bu tür hatalara *gecikmiş hata* denir ve ayıklaması en pahalı hata türüdür.

2.2. Aynı İstekler, Kapsüllenmiş Sınıfta

Şimdi aynı üç isteği, alanları `private` olan ve kuralı içinde barındıran bir `Ogrenci` sınıfına gönderelim. Sınıfın tam listesini Bölüm 3'te inceleyeceğiz; şimdilik yalnızca davranışına bakalım.

OgrenciDemo.java

```
1          System.out.println("Baslangic : " + o);
2
3          System.out.println();
4          System.out.println("--- Ayni hatali atamalar deneniyor ---");
5          o.setYas(-3);
6          o.setAd("");
7          o.notEkle(350.0);
8          System.out.println("Sonra     : " + o);
9
10         System.out.println();
11         System.out.println("--- Gecerli islemler ---");
12         o.notEkle(78.0);
13         o.notEkle(64.0);
14         o.setYas(21);
15         System.out.printf("Ortalama  : %.2f (%d not)%n", o.getNotOrtalamasi(),
16                 o.getNotAdedi());
```

Kod 4.3 — Kapsüllenmiş sınıfa gönderilen aynı istekler

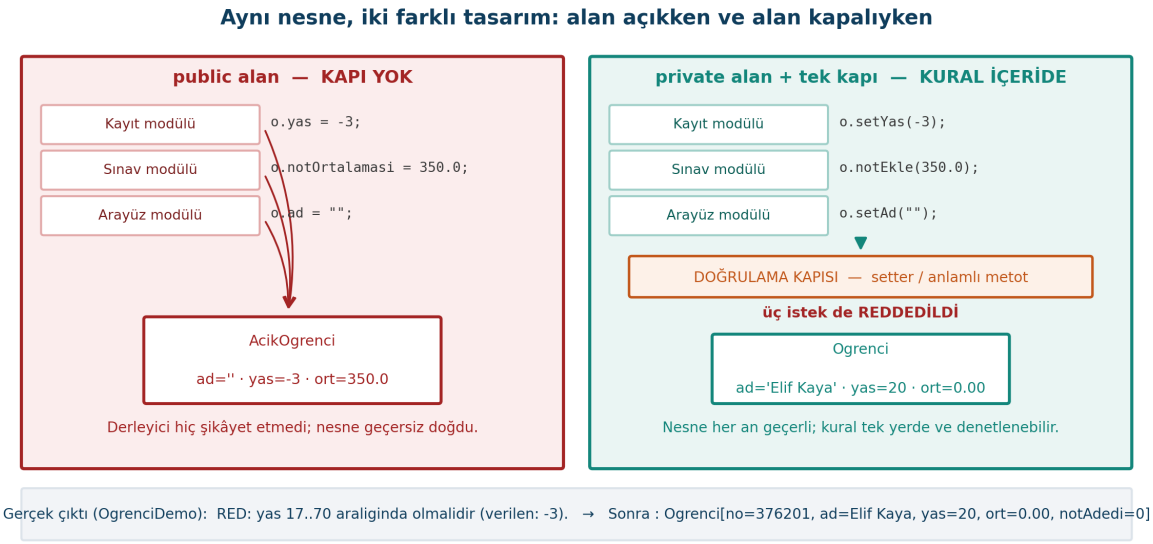
```
▸ TERMINAL — java OgrenciDemo (ilk bölüm)
Baslangic : Ogrenci[no=376201, ad=Elif Kaya, yas=20, ort=0.00, notAdedi=0]

--- Ayni hatali atamalar deneniyor ---
  RED: yas 17..70 araliginda olmalidir (verilen: -3).
  RED: ad bos birakilamaz; eski deger korundu.
```

```
  RED: not 0..100 araliginda olmalidir (verilen: 350.0).
Sonra     : Ogrenci[no=376201, ad=Elif Kaya, yas=20, ort=0.00, notAdedi=0]

--- Gecerli islemler ---
Ortalama  : 71.00 (2 not)
```

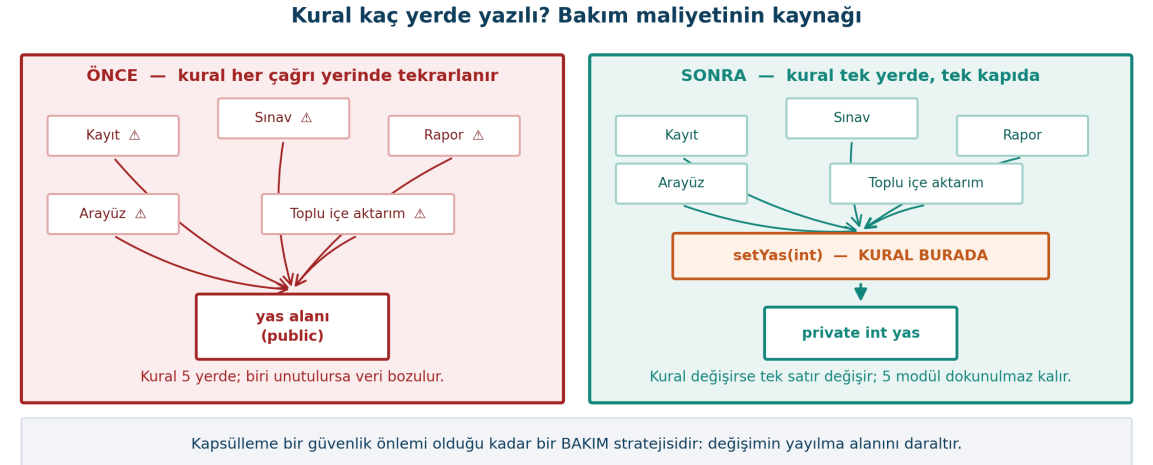
Üç istek de **reddedildi** ve nesne ilk geçerli durumunu korudu: `ad=Elif Kaya`, `yas=20`. Aradaki farkı üreten şey bir teknik hüner değildir; sorumluluğun yer değiştirmesidir. Önceki tasarımda kuralı **çağıran** bilmek zorundaydı; yeni tasarımda kuralı **sınıf** bilir.



Şekil 4.1 — Aynı nesne iki farklı tasarımda: alan açıkken ve alan kapalıyken

2.3. Sorunun Asıl Boyutu: Bakım

Kapsülleme çoğu zaman bir "güvenlik" konusu gibi anlatılır. Bu eksik bir anlatımdır. Asıl kazanç **bakımdadır**. Yaş kuralının 17–70 değil de 16–75 olması gerektiğini düşünelim. Açık alanlı tasarımda bu kuralı yazan her modül tek tek bulunup güncellenmelidir; biri unutulursa veri yeniden bozulur. Kapsüllenmiş tasarımda değişecek yer **tek bir satırdır**.



Şekil 4.2 — Kuralın kaç yerde yazılı olduğu, bakım maliyetini doğrudan belirler

Ölçüt	Alan `public` iken	Alan `private` + kapı iken
Kuralın yazıldığı yer sayısı	Her çağrı yerinde (N adet)	Tek yerde (1 adet)
Kural değişince dokunulacak dosya	N dosya	1 dosya
Kuralın unutulma olasılığı	Her yeni çağrı yerinde var	Yok — kapıdan geçmek zorunlu
Geçersiz nesne üretilebilir mi?	Evet	Hayır
Hatanın oluştuğu yer belli mi?	Hayır (gecikmiş hata)	Evet (anında ret iletisi)
Alanın iç gösterimini değiştirebilir misiniz?	Hayır — tüm kod kırılır	Evet — kapı aynı kaldıkça serbest

Tablo 4.2 — Kapsülleme kararının somut sonuçları

> ✓ **İPUCU**
>
> Son satır çoğu zaman gözden kaçar ama en değerli maddedir: `notOrtalamasi` alanını silip yerine `notToplami` ve `notAdedi` koyabildik; çünkü dışarısı yalnızca `getNotOrtalamasi()` kapısını biliyordu. Açık alanlı tasarımda bu değişiklik **mümkün değildi**.

3. Kapsülleme: Tanım ve Sözdizimi

Kapsülleme (encapsulation) Bir nesnenin verisini dış dünyadan gizleyip, o veriye erişimi yalnızca sınıfın kendi tanımladığı **denetimli kapılardan** sağlayan tasarım ilkesi. Veri ile o veriyi geçerli tutan kural aynı türün içinde birlikte bulunur.

Erişim belirleyici (access modifier) Bir sınıfın, alanın, metodun veya yapıcının **nereden görülebileceğini** belirleyen anahtar sözcük. Java'da dört düzey vardır: `private`, varsayılan (paket), `protected`, `public`.

Getter (erişimci) Bir alanın değerini dışarıya **okutan** metot. Java geleneğinde adı `getAlanAdi()` biçimindedir; `boolean` alanlarda `isAlanAdi()` kullanılır.

Setter (değiştirici) Bir alanın değerini dışarıdan **değiştirmeye** aracılık eden metot. Adı `setAlanAdi(...)` biçimindedir. Kapsüllemenin anlam kazandığı yer, setter'ın içindeki **doğrulama** kodudur.

Değişmez (bağışıklık kazanmış / immutable) nesne Oluşturulduktan sonra durumu hiçbir biçimde değiştirilemeyen nesne. Değişiklik gerektiğinde mevcut nesne değil, **yeni bir nesne** üretilir.

3.1. En Küçük Kapsülleme: Alanı Kapat, Kapıyı Aç

Kapsüllemenin en yalın hâli iki adımdan oluşur: alanın önüne `private` yazmak ve gerekiyorsa bir getter/setter çifti eklemek.

Ogrenci.java

```
1  public class Ogrenci {
2
3      public static final int EN_KUCUK_YAS = 17;
4      public static final int EN_BUYUK_YAS = 70;
5
6      private final String ogrenciNo;   // dogumda belirlenir, bir daha degismez
7      private String ad;
8      private int yas;
9      private double notToplami;
10     private int notAdedi;
11
12     public Ogrenci(String ogrenciNo, String ad, int yas) {
13         this.ogrenciNo = ogrenciNo;
14         setAd(ad);          // dogrulama tek yerde: setter'da
15         setYas(yas);
16         this.notToplami = 0.0;
17         this.notAdedi = 0;
18     }
```

Kod 4.4 — Alanların tanımı ve yapıcı

Satır satır çözümleyelim (satır numaraları **yukarıdaki kod listesine** göredir, dosyadaki mutlak numaralara değil):

1. **3–4.** `public static final` sabitler. `static` konusu Hafta 6'nın işidir; şimdilik "bu iki sayı sınıfın tamamına ait, tek kopya ve değişmez" demek yeterlidir. Kuralın **sihirli sayı** olarak koda gömülmemesi iyi bir alışkanlıktır.

2. **6.** `private final String ogrenciNo;` — hem gizli hem salt-okunur. Bir öğrencinin numarası doğumda belirlenir ve bir daha değişmez.
3. **7–10.** Diğer alanların hepsi `private`. Dışarıdan hiçbir kod bu adları göremez, dolayısıyla yazamaz.
4. **14–15.** Yapıcı, alanlara **doğrudan atama yapmak yerine setter'ları çağırıyor**. Böylece doğrulama kuralı yapıcıda ikinci kez yazılmamış olur. Bu, kod tekrarını önleyen küçük ama önemli bir alışkanlıktır.

3.2. Getter: Okuma Kapısı

Ogrenci.java

```
1       // ---- salt-okunur alan: setter YOK ----
2       public String getOgrenciNo() {
3           return ogrenciNo;
4       }
5
6       // ---- dogrulamali setter ----
7       public String getAd() {
8           return ad;
9       }
```

Kod 4.5 — Salt-okunur alan ve sıradan getter

`getOgrenciNo()` bir getter'dır ama karşılığında bir `setOgrenciNo()` **yoktur**. Bu bilinçli bir tercihtir: alan dışarıya okunur, ama yazma yolu hiç açılmamıştır. Kapsüllemede kapılar çift çift açılmak zorunda değildir.

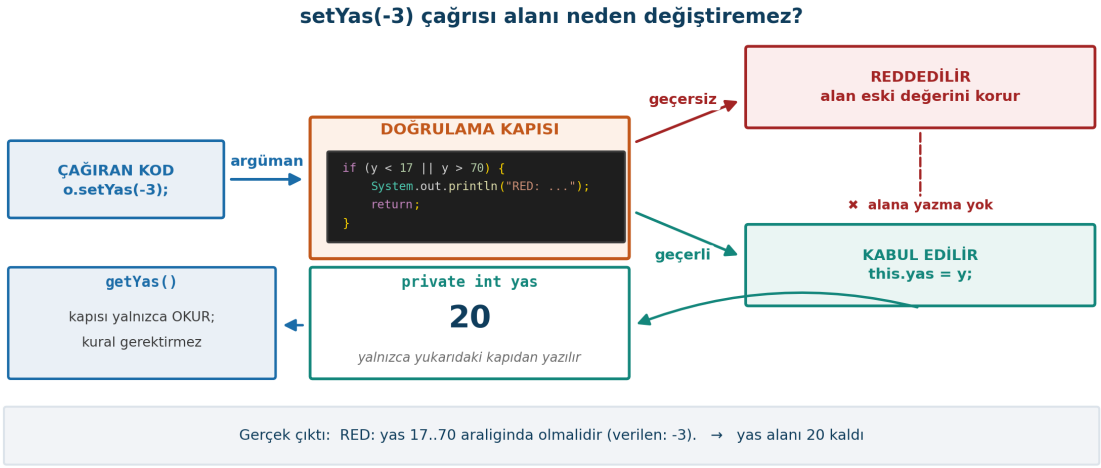
3.3. Setter: Denetimli Yazma Kapısı

Ogrenci.java

```
1       public int getYas() {
2           return yas;
3       }
4
5       public void setYas(int yeniYas) {
6           if (yeniYas < EN_KUCUK_YAS || yeniYas > EN_BUYUK_YAS) {
7               System.out.println("  RED: yas " + EN_KUCUK_YAS + ".." + EN_BUYUK_YAS
8                       + " araliginda olmalidir (verilen: " + yeniYas + ").");
9               return;
10          }
11          this.yas = yeniYas;
12      }
```

Kod 4.6 — Doğrulamalı setter

Bu metot, kapsüllemenin özüdür. Geçersiz bir değer geldiğinde iki şey yapar: bir uyarı iletisi yazar ve **atamayı yapmadan** geri döner. Sonuç olarak alan eski geçerli değerini korur. Setter'ın gövdesi bir **kapı bekçisidir**; atama satırına ancak koşulu geçen değerler ulaşır.



Şekil 4.3 — Bir setter çağrısının doğrulama kapısından geçişi

> **▲ DİKKAT**
>
> Setter'ın geçersiz değeri nasıl karşılayacağı bir **tasarım kararıdır**. Bu haftaki örneklerde sessizce reddedip uyarı yazıyoruz; çünkü henüz istisna (exception) mekanizmasını görmedik. Gerçek projelerde tercih edilen yol, `IllegalArgumentException` fırlatmaktır. Bu yolu Hafta 11'de öğreneceğiz; `ParaBirimi` örneğinde bir ön gösterimini göreceksiniz.

3.4. Türetilmiş Getter: Alanı Olmayan Kapı

Bir getter'ın arkasında **mutlaka bir alan olması gerekmez**. `Ogrenci` sınıfında `notOrtalamasi` diye bir alan yoktur; ortalama, iki alandan her çağrıda hesaplanır.

Ogrenci.java

```
    // ---- turetilmis (hesaplanan) getter: karsiligi olan alan YOK ----
    public double getNotOrtalamasi() {
        if (notAdedi == 0) {
            return 0.0;
        }
        return notToplami / notAdedi;
    }

    public int getNotAdedi() {
        return notAdedi;
    }

    /* Ortalama disaridan atanamaz; yalnizca gecerli not eklenerek degisir. */
    public boolean notEkle(double puan) {
        if (puan < 0.0 || puan > 100.0) {
            System.out.println("  RED: not 0..100 araliginda olmalidir (verilen: "
                    + puan + ").");
            return false;
        }
        notToplami += puan;
        notAdedi++;
        return true;
```

```
23      }
```

Kod 4.7 — Türetilmiş getter ve durumu değiştiren anlamlı metot

Bu tasarımın iki kazancı vardır. Birincisi, ortalama ile not listesi **asla çelişemez**; çünkü tek bir doğruluk kaynağı vardır. İkincisi, ortalamayı dışarıdan atamak imkânsızdır: `setNotOrtalamasi()` yoktur, yalnızca `notEkle()` vardır ve o da yalnızca 0–100 aralığındaki notları kabul eder.

```
▸ TERMINAL — java OgrenciDemo (ikinci bölüm)
Son durum : Ogrenci[no=376201, ad=Elif Kaya, yas=21, ort=71.00, notAdedi=2]

--- Salt-okunur alan ---
Ogrenci no okunabilir : 376201
Yazma yolu ise hic acilmamistir.
```

> **ⓘ BİLGİ**
>
> `notEkle()` bir setter değildir; bir **iş metodudur**. Adı, gerçekleşen olayı anlatır (“nota bir not eklendi”), bir alanın adını değil. Kapsüllemede iyi tasarım, alan adlarını değil **olayları** dışarıya açar. Bu ayrımı Bölüm 4.5'te derinleştireceğiz.

3.5. Sınıfın Tam Listesi

`Ogrenci` sınıfının dört alanı vardır ve dördü de farklı bir kapı düzenine sahiptir. Aşağıdaki tablo sınıfın dışa açık **sözleşmesini** özetler.

Alan	Erişim	Okuma kapısı	Yazma kapısı	Gerekçe
`ogrenciNo`	`private final`	`getOgrenciNo()`	**yok**	Kimlik doğumda belirlenir, değişmez
`ad`	`private`	`getAd()`	`setAd()` — boş olamaz	Değişebilir ama anlamlı olmalı
`yas`	`private`	`getYas()`	`setYas()` — 17..70	Değişebilir ama fiziksel sınırı var
`notToplami` / `notAdedi`	`private`	`getNotOrtalamasi()` (türetilmiş)	`notEkle()` — 0..100	Doğrudan atanamaz; ancak olayla değişir

Tablo 4.3 — `Ogrenci` sınıfının dışa açık sözleşmesi

4. Derinleşme

4.1. Dört Erişim Düzeyi

Java'da erişim dört düzeydedir. Üçünün anahtar sözcüğü vardır; dördüncüsü **hiçbir şey yazmayarak** elde edilir ve “varsayılan” ya da “paket-özel” adıyla anılır.



Şekil 4.4 — Dört erişim düzeyinin görünürlük matrisi

Belirteç	Aynı sınıf	Aynı paket	Alt sınıf (başka paket)	Herkes	Tipik kullanım
`private`	✓	✗	✗	✗	Alanlar ve yardımcı metotlar
(yazılmaz)	✓	✓	✗	✗	Paket içi işbirliği yapan sınıflar
`protected`	✓	✓	✓	✗	Alt sınıfa açılan yapı taşları
`public`	✓	✓	✓	✓	Sözleşmenin dışa açık yüzü

Tablo 4.4 — Erişim düzeyleri ve tipik kullanım alanları

Tabloda dikkat edilmesi gereken iki incelik vardır:

1. `protected`, adının çağrıştırdığının aksine `private`'dan **daha geniştir**; paket içindeki herkese açıktır. Yani `protected` bir alan, aynı paketteki tamamen ilgisiz bir sınıf tarafından da görülür.
2. `protected` üye, başka paketteki alt sınıfa yalnızca **kendi üzerinden** (`this.alan`) açıktır. Alt sınıf, üst sınıftan üretilmiş **başka bir nesnenin** `protected` alanına erişemez.

4.2. Paket Kavramı ve Erişim Sınırı

Paket (package), sınıfları mantıksal olarak gruplayan ve dosya sisteminde klasörlere karşılık gelen yapıdır. C dilinde birden fazla dosyayı bir başlık dosyasıyla gruplarız; Java'da bu iş `package` bildirimiyle yapılır ve **erişim düzeylerinin sınırını** da paketler çizer.

```java
// okul/Kayit.java
package okul;

public class Kayit {

    public    String kurum   = "KTU Of Teknoloji Fakultesi";  // her yerden
    protected String bolum   = "Yazilim Muhendisligi";        // paket + alt sinif
              String donem   = "2026-2027 Guz";               // yalnizca paket
    private   String gizliId = "K-000731";                    // yalnizca bu sinif

    public String getGizliId() {
        return gizliId;    // private alana ancak sinifin kendi metodu erisir
    }
}
```

Kod 4.8 — Dört erişim düzeyini bir arada barındıran sınıf

Listenin 7. satırında `donem` alanının önünde hiçbir belirteç yoktur. Bu, “varsayılan” ya da “paket-özel” erişimdir ve `public`'in kısaltması **değildir** — tam tersine, `protected`'tan daha dardır.

Aynı paketten bakış

```java
// okul/PaketIciDemo.java
package okul;

public class PaketIciDemo {

    public static void main(String[] args) {
        Kayit k = new Kayit();
        System.out.println("=== Ayni paket: okul.PaketIciDemo ===");
        System.out.println("  public    kurum = " + k.kurum);
        System.out.println("  protected bolum = " + k.bolum);
        System.out.println("  varsayilan donem = " + k.donem);
        // System.out.println(k.gizliId);   // DERLEME HATASI: private
        System.out.println("  private   gizliId -> yalnizca getGizliId(): "
                + k.getGizliId());
        System.out.println();
        k.hepsiniYaz();
    }
}
```

Kod 4.9 — Aynı paketteki sınıf üç düzeye de erişir

```
▸ TERMINAL — java okul.PaketIciDemo
=== Ayni paket: okul.PaketIciDemo ===
  public    kurum = KTU Of Teknoloji Fakultesi
  protected bolum = Yazilim Muhendisligi
  varsayilan donem = 2026-2027 Guz
  private   gizliId -> yalnizca getGizliId(): K-000731

  [Kayit icinden] kurum   = KTU Of Teknoloji Fakultesi
  [Kayit icinden] bolum   = Yazilim Muhendisligi
  [Kayit icinden] donem   = 2026-2027 Guz
  [Kayit icinden] gizliId = K-000731
```

Farklı paketten bakış

disarida/PaketDisiDemo.java

```
 */
package disarida;

import okul.Kayit;

public class PaketDisiDemo extends Kayit {

    public void kendiUzerindenOku() {
        System.out.println("=== Farkli paket, ALT SINIF: disarida.PaketDisiDemo ===");
        System.out.println("  public    kurum = " + this.kurum);
        System.out.println("  protected bolum = " + this.bolum + "   <- miras yoluyla");
        // System.out.println(this.donem);     // DERLEME HATASI: varsayilan erisim
        // System.out.println(this.gizliId);   // DERLEME HATASI: private
        System.out.println("  varsayilan donem -> ERISILEMEZ (baska paket)");
        System.out.println("  private   gizliId -> ERISILEMEZ");
    }

    public static void main(String[] args) {
        new PaketDisiDemo().kendiUzerindenOku();

        System.out.println();
        System.out.println("=== Farkli paket, ILGISIZ NESNE uzerinden ===");
        Kayit baskasi = new Kayit();
        System.out.println("  public    kurum = " + baskasi.kurum);
        // System.out.println(baskasi.bolum);  // DERLEME HATASI: protected, farkli paket
        System.out.println("  protected bolum -> ERISILEMEZ (baska nesne, baska paket)");
        System.out.println("  Yalnizca public uyeler disari acik kalir.");
    }
}
```

Kod 4.10 — Farklı paketteki alt sınıfın erişim sınırları

```
▸ TERMINAL — java disarida.PaketDisiDemo
=== Farkli paket, ALT SINIF: disarida.PaketDisiDemo ===
  public    kurum = KTU Of Teknoloji Fakultesi
  protected bolum = Yazilim Muhendisligi   <- miras yoluyla
  varsayilan donem -> ERISILEMEZ (baska paket)
  private   gizliId -> ERISILEMEZ

=== Farkli paket, ILGISIZ NESNE uzerinden ===
  public    kurum = KTU Of Teknoloji Fakultesi
  protected bolum -> ERISILEMEZ (baska nesne, baska paket)
  Yalnizca public uyeler disari acik kalir.
```

Çıktının ikinci bölümü, yukarıda değindiğimiz inceliği somutlaştırır: `PaketDisiDemo` kendi `bolum` alanını (miras yoluyla edindiği alanı) okuyabilir; ama `new Kayit()` ile ürettiği **başka bir nesnenin** aynı alanına erişemez. `protected`, miras ilişkisine bağlı bir ayrıcalıktır; genel bir açıklık değildir.



Şekil 4.5 — İki paket üzerinden erişim düzeylerinin somut gösterimi

> **ⓘ BİLGİ**
>
> Bu haftaki paket örneği, `protected`'ı **tanımak** içindir. `extends`, `super` ve mirasın tüm ayrıntıları Hafta 7'nin konusudur; orada `protected`'ı bir kez daha, bu kez miras bağlamında ele alacağız.

4.3. `final` Alan ve Salt-Okunur Tasarım

`final` bir alan, kendisine **bir kez** değer atanmasına izin verir. Bu atama ya bildirim satırında ya da yapıcıda yapılmalıdır; yapıcı tamamlandığında `final` alanların hepsi atanmış olmak zorundadır.

Yazım	Anlamı	İkinci atama
`final int x = 5;`	Bildirimde atandı	Derleme hatası
`final int x;` + yapıcıda atama	Yapıcıda atandı	Derleme hatası
`final int x;` (hiç atanmadı)	Eksik başlatma	Derleme hatası
`final int[] a = {1,2};` + `a[0]=9;`	Bağ değişmedi, içerik değişti	**Derlenir** — dikkat!

Tablo 4.5 — `final` alanın atama kuralları



Şekil 4.6 — `final` bağı dondurur, nesne içeriğini dondurmaz

> **▲ DİKKAT**
>
> `final`, işaret edilen **nesnenin** değişmezliğini garanti etmez; yalnızca değişkenin **başka bir nesneyi işaret etmesini** engeller. Bir `final int[]` alanın elemanları serbestçe değiştirilebilir. İçeriğin de korunması isteniyorsa değişmez (immutable) tasarım gerekir — Bölüm 4.4.

Salt-okunur bir alan üç adımda kurulur: alan `private final` yapılır, değer yalnızca yapıcıda atanır, dışarıya yalnızca getter açılır. `Ogrenci.ogrenciNo` ve `BankaHesabi.iban` bu düzendedir.

4.4. Değişmez (Immutable) Sınıf

Kapsüllemenin en katı biçimi, alanların yalnızca gizlenmesi değil, **hiç değiştirilememesidir**. Böyle sınıflara değişmez (immutable) sınıf denir. Java'nın `String`, `Integer` ve `LocalDate` sınıfları bu düzendedir.

#	Kural	Neden gerekli?
1	Sınıf `final` yapılır	Alt sınıf, ezme yoluyla değişmezliği delemesin
2	Tüm alanlar `private final`	Ne dışarıdan görülsün ne de yeniden atansın
3	Hiç setter yazılmaz	Durumu değiştirecek tek kapı bile kalmasın
4	Dönüşüm metotları **yeni nesne** döndürür	Değişiklik isteği, mevcut nesneyi bozmadan karşılansın
5	Nesne türü alan varsa kopyası saklanır/verilir	Dışarıdaki referans üzerinden içerik değiştirilemesin

Tablo 4.6 — Değişmez sınıf tasarımının kuralları

ParaBirimi.java

```java
public final class ParaBirimi {

    private final long kurus;      // tam sayi: kayan nokta yuvarlama hatasi yok
    private final String kod;      // "TRY", "USD", ...

    public ParaBirimi(long kurus, String kod) {
        if (kod == null || kod.length() != 3) {
```

```
            throw new IllegalArgumentException("Para birimi kodu 3 harf olmalidir.");
        }
        this.kurus = kurus;
        this.kod = kod.toUpperCase();
    }

    public static ParaBirimi lira(double tutar) {
        return new ParaBirimi(Math.round(tutar * 100.0), "TRY");
    }
```

Kod 4.11 — Değişmez sınıfın iskeleti

Listenin 1. satırındaki `final class`, 3–4. satırlarındaki `private final` alanlar ve sınıfın tamamında **hiç setter bulunmayışı** ilk üç kuralı karşılar. Dördüncü kural, dönüşüm metotlarında görülür:

ParaBirimi.java

```
    /* Degistirmez; YENI nesne dondurur. */
    public ParaBirimi topla(ParaBirimi digeri) {
        birimKontrol(digeri);
        return new ParaBirimi(this.kurus + digeri.kurus, this.kod);
    }

    public ParaBirimi cikar(ParaBirimi digeri) {
        birimKontrol(digeri);
        return new ParaBirimi(this.kurus - digeri.kurus, this.kod);
    }

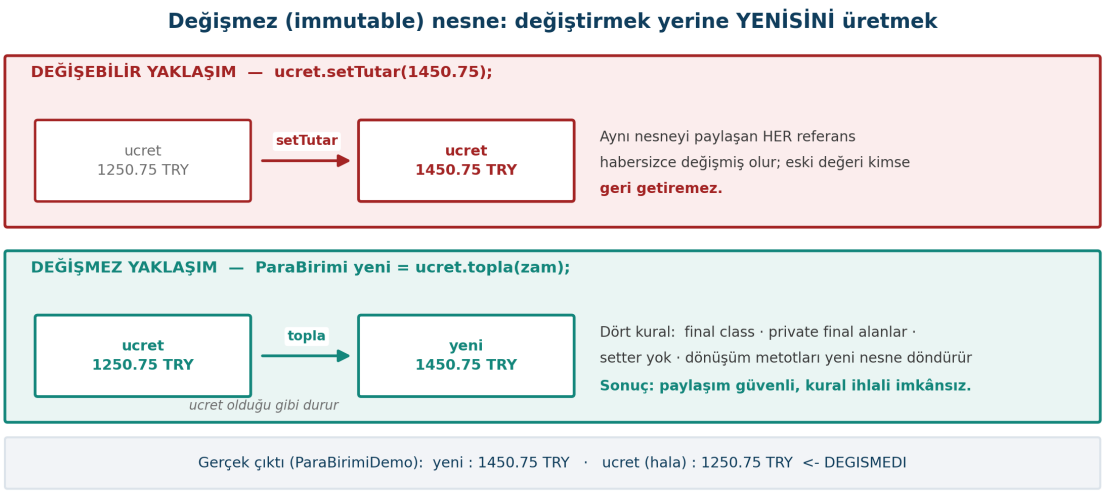
    public ParaBirimi carp(int katsayi) {
        return new ParaBirimi(this.kurus * katsayi, this.kod);
    }

    private void birimKontrol(ParaBirimi digeri) {
        if (!this.kod.equals(digeri.kod)) {
            throw new IllegalArgumentException(
                    "Farkli para birimleri toplanamaz: " + this.kod + " / " + digeri.kod);
        }
    }
```

Kod 4.12 — Dönüşüm metotları nesneyi değiştirmez, yenisini üretir

```
▸ TERMINAL — java ParaBirimiDemo (ilk bölüm)
ucret          : 1250.75 TRY
zam            : 200.00 TRY
yeni           : 1450.75 TRY
ucret (hala)   : 1250.75 TRY   <- DEGISMEDI

--- Paylasim guvenli ---
paylasilan     : 1250.75 TRY
  yardimci icinde uretilen: 2501.50 TRY
cagri sonrasi  : 1250.75 TRY   <- DEGISMEDI
```



Şekil 4.7 — Değişmez nesnede değişiklik, yeni nesne üretmek demektir

Değişmezliğin bedeli, her işlemde yeni bir nesne üretilmesidir. Kazancı ise büyüktür: böyle bir nesne serbestçe paylaşılabilir, bir metoda parametre olarak verilebilir, koleksiyonlarda anahtar olarak kullanılabilir — hiçbir durumda arkadan değiştirilme riski yoktur.

> **✓ İPUCU**
>
> `ParaBirimi` sınıfında tutar `double` yerine `long kurus` olarak saklanır. Bu, kapsüllemenin bir başka kazancını gösterir: **iç gösterim** dışarıdan gizlidir. `getTutar()` çağıran kod, arkada kuruş cinsinden tam sayı tutulduğunu bilmek zorunda değildir.

4.5. Setter mi, Anlamlı Metot mu?

Her değişiklik bir "alan ataması" değildir; çoğu zaman bir **iş olayıdır**. Bir banka hesabında bakiyenin değişmesi bir atama değil, para yatırma ya da çekme olayıdır. Bu ayrımı koda taşımak, sınıfın anlaşılabilirliğini kökten değiştirir.

Setter yaklaşımı	Anlamlı metot yaklaşımı
`h.setBakiye(h.getBakiye() + 500);`	`h.yatir(500);`
Kural çağıranda: "önce oku, topla, yaz"	Kural sınıfta: tek çağrı
Günlük limit denetimi yapılamaz	Günlük limit metodun içinde
İki eşzamanlı çağrı birbirini bozabilir	İşlem bütünlüğü sınıfta korunur
Ne olduğu koddan anlaşılmaz	Kod, iş olayını anlatır

Tablo 4.7 — Aynı işin iki farklı tasarımı

BankaHesabi.java

```
1      /* Yalnizca OKUMA. setBakiye() BILEREK yazilmamistir. */
2      public double getBakiye() {
3          return bakiye;
4      }
```

```java

    public double getKalanGunlukHak() {
        return GUNLUK_CEKIM_LIMITI - bugunCekilen;
    }

    public boolean yatir(double tutar) {
        if (tutar <= 0) {
            System.out.println("  RED: yatirilan tutar pozitif olmalidir.");
            return false;
        }
        bakiye += tutar;
        System.out.printf("  OK : %.2f yatirildi. Bakiye = %.2f%n", tutar, bakiye);
        return true;
    }
```

Kod 4.13 — Yalnızca okuma kapısı ve anlamlı bir iş metodu

BankaHesabi.java

```java
    public boolean cek(double tutar) {
        if (tutar <= 0) {
            System.out.println("  RED: cekilen tutar pozitif olmalidir.");
            return false;
        }
        if (tutar > bakiye) {
            System.out.printf("  RED: yetersiz bakiye (%.2f > %.2f).%n", tutar, bakiye);
            return false;
        }
        if (bugunCekilen + tutar > GUNLUK_CEKIM_LIMITI) {
            System.out.printf("  RED: gunluk limit asilir (%.2f + %.2f > %.2f).%n",
                    tutar, bugunCekilen, GUNLUK_CEKIM_LIMITI);
            return false;
        }
        bakiye -= tutar;
        bugunCekilen += tutar;
        System.out.printf("  OK : %.2f cekildi. Bakiye = %.2f%n", tutar, bakiye);
        return true;
    }
```

Kod 4.14 — Üç iş kuralını birlikte uygulayan `cek()` metodu

```
▸ TERMINAL — java BankaHesabiDemo (ret durumları)
  RED: yatirilan tutar pozitif olmalidir.
  RED: cekilen tutar pozitif olmalidir.
  RED: yetersiz bakiye (99999.00 > 1400.00).

--- Gunluk limit ---
  OK : 5000.00 yatirildi. Bakiye = 6400.00
  OK : 1500.00 cekildi. Bakiye = 4900.00
  RED: gunluk limit asilir (300.00 + 1800.00 > 2000.00).
Kalan gunluk hak: 200.00

Son durum: Hesap[TR33 0006 1005, Ayse Yilmaz, bakiye=4900.00]
```

`cek()` metodu üç ayrı kuralı sırayla denetler: tutar pozitif mi, bakiye yeterli mi, günlük limit aşılıyor mu? Bu üç kural birlikte anlam ifade eder ve **hiçbir setter** onları bir arada uygulayamazdı. Bu, kapsüllemenin "kuralı içeri taşıma" yüzüdür.



Şekil 4.8 — Bir alan için hangi kapının açılacağına karar verme akışı

4.6. Aynı Bilgiyi İki Alanda Tutmamak

Kapsüllemenin sessiz düşmanı **yedekli veridir**. Aynı bilgiyi iki alanda tutmak, o iki alanı sürekli eşitleme borcu doğurur; bir güncelleme unutulduğunda nesne tutarsız hâle gelir.

Sicaklik.java

```
1     public void setCelsius(double yeni) {
2         if (yeni < MUTLAK_SIFIR) {
3             System.out.printf("  RED: %.2f C mutlak sifirin altinda; deger degismedi.%n",
4                     yeni);
5             return;
6         }
7         this.celsius = yeni;
8         if (yeni < enDusuk) {
9             enDusuk = yeni;
10        }
11        if (yeni > enYuksek) {
12            enYuksek = yeni;
13        }
14    }
15
16    /* Fahrenheit ve Kelvin icin ALAN YOKTUR; ihtiyac aninda hesaplanir.
17       Boylece iki alanin birbiriyle celismesi imkansiz hale gelir. */
18    public double getFahrenheit() {
19        return celsius * 9.0 / 5.0 + 32.0;
20    }
21
22    public double getKelvin() {
23        return celsius - MUTLAK_SIFIR;
24    }
25
26    public void setFahrenheit(double f) {
27        setCelsius((f - 32.0) * 5.0 / 9.0);
28    }
```

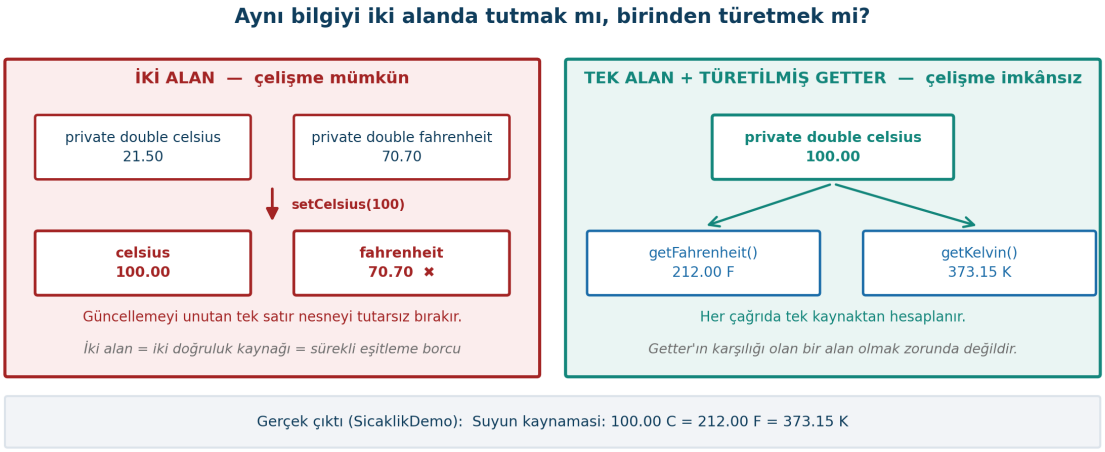
Kod 4.15 — Doğrulamalı setter ve türetilmiş getter'lar

```
▸ TERMINAL — java SicaklikDemo
Baslangic     : 21.50 C = 70.70 F = 294.65 K

--- Gecersiz deger ---
  RED: -300.00 C mutlak sifirin altinda; deger degismedi.
Deger degismedi: 21.50 C = 70.70 F = 294.65 K

--- Fahrenheit uzerinden yazma ---
Suyun kaynamasi: 100.00 C = 212.00 F = 373.15 K
Soguk gun      : -40.00 C = -40.00 F = 233.15 K

Gorulen en dusuk : -40.00 C
Gorulen en yuksek: 100.00 C
```



Şekil 4.9 — Tek doğruluk kaynağı: Fahrenheit ve Kelvin için alan tutulmaz

`Sicaklik` sınıfında yalnızca `celsius` alanı vardır. Fahrenheit ve Kelvin her çağrıda ondan hesaplanır; dolayısıyla üçü **asla çelişemez**. `setFahrenheit()` bile kendi başına atama yapmaz, çevirip `setCelsius()` çağırır — böylece mutlak sıfır kuralı tek bir yerde kalır.

✖ SIK YAPILAN HATA

Yedekli alan, gerçek projelerde en sık karşılaşılan tutarsızlık kaynağıdır: `dogumTarihi` yanında `yas` tutmak, `birimFiyat` ve `adet` yanında `toplam` tutmak gibi. Kural basittir: **türetilebilen bilgi alanda tutulmaz, getter'da hesaplanır.**

5. C'den Java'ya Köprü

⇄ C DİLİNDEN JAVA'YA KÖPRÜ

C dilinde de veri gizleme vardır; ama gizlemenin birimi **dosya**, Java'da ise **tür**tür. C'de `static` anahtar sözcüğü bir değişkeni ya da fonksiyonu çeviri birimine (dosyaya) hapseder; dışarıya açılacak olanlar `.h` başlık dosyasında ilan edilir. Java'da bu iş, veri ile kuralı aynı sınıfın içine koyarak yapılır.



Şekil 4.10 — C'de gizleme dosya düzeyinde, Java'da tür düzeyindedir

Konu	C dilinde	Java'da
Gizleme birimi	Çeviri birimi (`.c` dosyası)	Sınıf (tür)
Gizleme aracı	`static` (dosya kapsamı)	`private`
Dışa açık sözleşme	`.h` başlık dosyasındaki ilanlar	`public` üyeler
Ara düzey (grup içi)	Yok (ya açık ya kapalı)	Varsayılan (paket) erişim
Miras için ayrıcalık	Yok	`protected`
Veri–fonksiyon bağı	Adlandırma geleneği (`hesap_yatir`)	Dilin kendisi (`hesap.yatir()`)
Değişmezlik	`const` (kısmî güvence)	`final` + değişmez tasarım
Doğrulama yeri	Her fonksiyonun başında elle	Setter / iş metodu içinde

Tablo 4.8 — C'deki veri gizleme ile Java kapsüllemesinin karşılaştırması

C'de disiplinli bir programcı da veriyi gizleyebilir: `struct` tanımını `.c` dosyasında tutup `.h` dosyasında yalnızca `typedef struct Hesap Hesap;` ilan etmek (opak işaretçi deseni) bunun klasik yoludur. Aradaki fark, Java'da bu disiplinin **dilin kendisi tarafından zorunlu kılınmasıdır**: `private` bir alana erişme girişimi derleme hatasıdır, geleneğe uymamak değil.

ⓘ BİLGİ

C++ bilenler için not: C++'ta bu sözcükler **etiket** biçiminde (`private:`) kullanılır ve bir sonraki etikete kadar geçerlidir; Java'da her üyenin belirteci ayrı yazılır. Ayrıca C++'ın `friend` mekanizmasının Java'da karşılığı

yoktur.

6. Sık Yapılan Hatalar

6.1. Düşünmeden Yazılan Getter/Setter Çifti

✖ HATA 1 — Kapsülleme kılığında açık alan

Her alana otomatik olarak bir getter ve bir setter üretmek kapsülleme **değildir**. Bu, alanları `public` bırakmanın daha uzun yazılmış hâlidir: dışarısı yine istediği değeri yazabilmektedir.

```
// YANLIS - kapsulleme goruntusu, kapsulleme yok
public class Ogrenci {
    private int yas;
    public int getYas() { return yas; }
    public void setYas(int y) { this.yas = y; }   // hicbir kural yok
}
// o.setYas(-3);  ->  yine -3 olur

// DOGRU - kural kapinin uzerinde
public class Ogrenci {
    private int yas;
    public int getYas() { return yas; }
    public void setYas(int y) {
        if (y < 17 || y > 70) { return; }         // gecersiz deger reddedilir
        this.yas = y;
    }
}
```

Kod 4.16 — Kapsülleme görüntüsü ile gerçek kapsülleme

6.2. Aynı Sınıfın İçinden `private`'a Erişilemeyeceğini Sanmak

✖ HATA 2 — `private`'ın kapsamını yanlış anlamak

`private` üye, **sınıfın tamamına** açıktır; nesneye değil. Aynı sınıfın bir metodu, başka bir `Ogrenci` nesnesinin `private` alanına da erişebilir. `ParaBirimi.topla()` metodunda `digeri.kurus` yazımı tam olarak buna dayanır ve derlenir.

6.3. `final` ile Değişmezliği Karıştırmak

✖ HATA 3 — `final` alanı değişmez sanmak

`private final int[] notlar;` bildirimi dizinin **içeriğini** korumaz. `notlar[0] = 0;` satırı sorunsuz derlenir. Değişmez bir sınıf yazmak istiyorsanız Bölüm 4.4'teki beş kuralın tamamı gereklidir.

6.4. Getter'ın Değiştirilebilir Nesne Döndürmesi

✖ HATA 4 — Kapıdan içerinin anahtarını uzatmak

Bir getter, sınıfın içindeki değiştirilebilir bir nesnenin **referansını** döndürüyorsa kapsülleme delinmiş demektir. Çağıran kod, dönen referans üzerinden nesnenin içini değiştirebilir. Çözüm: ya değişmez bir tür döndürmek ya da kopyasını döndürmektir.

```
// YANLIS - ic dizinin referansi disari veriliyor
public int[] getNotlar() { return notlar; }
// int[] d = o.getNotlar();  d[0] = 999;   ->  nesnenin ici degisti

// DOGRU - kopya veriliyor
public int[] getNotlar() { return notlar.clone(); }
```

Kod 4.17 — Kapsüllemeyi delen getter ve düzeltilmiş hâli

6.5. Yapıcıda Doğrulamayı Atlamak

✖ HATA 5 — Geçersiz doğan nesne

Setter'a doğrulama koyup yapıcıda doğrudan atama yapmak, nesnenin **geçersiz doğmasına** izin verir. Bu yüzden `Ogrenci` yapıcısı `this.ad = ad;` yerine `setAd(ad);` çağırır.

6.6. Derleyici İletileri Sözlüğü

Derleyici iletisi	Ne demek?	Ne yapmalı?
`The field Ogrenci.yas is not visible`	`private` bir alana dışarıdan erişmeye çalıştınız	Getter/setter kullanın ya da erişim düzeyini gözden geçirin
`The final field ... cannot be assigned`	`final` alana ikinci kez atama yapılıyor	Atamayı yapıcıya taşıyın ya da `final`'ı kaldırın
`The blank final field ... may not have been initialized`	`final` alan yapıcıda atanmadan kaldı	Tüm yapıcı yollarında alana değer atayın
`The method setX(int) is undefined for the type Y`	Olmayan bir setter çağrıldı	Bu bilinçli bir tasarımsa doğru kapıyı (iş metodunu) kullanın
`The type Kayit is not visible`	Sınıfın kendisi `public` değil, başka paketten erişiliyor	Sınıfı `public` yapın ya da aynı pakete taşıyın
`The constructor ... is not visible`	Yapıcı `private` ya da paket-özel	Fabrika metodu kullanın (Hafta 5) ya da yapıcıyı `public` yapın

Tablo 4.9 — Kapsüllemeyle ilgili derleme hataları ve çözümleri

7. İyi Tasarım İlkeleri

Bu bölümdeki maddeler, dönem boyunca yazacağınız her sınıfta uygulamanız beklenen kod yazım standartlarıdır.

1. **En dar düzeyle başlayın.** Her üyeyi önce `private` yazın; gerekiyorsa genişletin. Genişletmek kolaydır, daraltmak kullanıcı kodunu kırar. Sınıfın dışa açık yüzü küçüldükçe ileride değiştirilebilme özgürlüğü büyür.
2. **Alanlar hemen hemen her zaman `private` olmalıdır.** İstisnası, `public static final` sabitlerdir (`GUNLUK_CEKIM_LIMITI` gibi).
3. **Getter yazmadan önce sorun:** dışarısı bu bilgiyi gerçekten görmek zorunda mı? Görmüyorsa kapı açmayın.
4. **Setter yazmadan önce sorun:** bu değişiklik bir alan ataması mı, yoksa bir iş olayı mı? İş olayıysa `setX` değil `yatir`, `notEkle`, `stokCik` gibi bir ad kullanın.
5. **Doğrulamayı tek yerde toplayın.** Yapıcı da setter'ı çağırsın; kural iki kez yazılmasın.
6. **Türetilebilen bilgiyi alanda tutmayın.** Getter'da hesaplayın; tutarsızlık ihtimalini kökten kaldırın.
7. **Değişmezliği varsayılan tercih yapın.** Bir sınıfın durumu değişmek zorunda değilse değişmez tasarlayın.
8. **Getter'ınız değiştirilebilir nesne döndürüyorsa kopya döndürün.**
9. **Sabitleri adlandırın.** Koda gömülü `17`, `70`, `2000.0` gibi sihirli sayılar yerine anlamlı adlı sabitler kullanın.

> ✓ **İPUCU**
>
> VS Code'da **Sağ tık → Source Action → Generate Getters and Setters** ile otomatik üretim yapılabilir; ancak üretilen setter'lar **boştur**, doğrulama içermez. Gerekmeyenleri silin, gerekenlere kural ekleyin.

7.1. Adlandırma Kuralları

Üye türü	Kural	Örnek
Alan	camelCase, ad öbeği	`notOrtalamasi`, `bugunCekilen`
Sabit	SCREAMING_SNAKE_CASE	`GUNLUK_CEKIM_LIMITI`, `MUTLAK_SIFIR`
Getter	`get` + Alan (PascalCase)	`getBakiye()`, `getAd()`
`boolean` getter	`is` / `has` ön eki	`isOduncte()`
Setter	`set` + Alan	`setYas(int)`, `setFiyat(double)`
İş metodu	Fiil — olayı anlatır	`yatir()`, `notEkle()`, `stokCik()`
Dönüşüm metodu (değişmez)	Fiil, yeni nesne döndürür	`topla()`, `otele()`, `olcekle()`
Paket	tümü küçük harf, tekil	`okul`, `kutuphane`, `disarida`

Tablo 4.10 — Kapsülleme bağlamında Java adlandırma gelenekleri

8. Bir Sonraki Haftaya Köprü

Bu hafta nesnenin **yaşarken** geçerli kalmasını sağladık. Ancak bir boşluk kaldı: nesnenin **doğduğu an**. Geçersiz bir ad geldiğinde `Ogrenci` nesnesi yine de `ISIMSIZ` adıyla doğdu; oysa böyle bir nesnenin hiç doğmaması gerekirdi.

Hafta 5'te yapıcıları derinlemesine ele alacağız. **Statik fabrika metodu**, yapıcıyı `private` yapıp nesne üretimini de kapsülleme altına almanın yoludur — bu haftanın ilkesinin nesne doğumuna uygulanmış hâli.

Bu haftaki kavram	Hafta 5'te nasıl genişleyecek?
`private` alan	`private` **yapıcı** — nesne üretimi de denetlenecek
Setter'daki doğrulama	Yapıcıdaki doğrulama ve nesnenin hiç doğmaması
`final` alan	Nesne başlatma sırası: alan → başlatma bloğu → yapıcı
Sabit üretimi (`Koordinat.BASLANGIC`)	Statik fabrika metodu deseni
Kaynak (dosya, bağlantı) tutan alanlar	`AutoCloseable` ve `try-with-resources` ile kapatma güvencesi

Tablo 4.11 — Hafta 4 kavramlarının Hafta 5'teki karşılıkları

Daha uzun vadede `protected` Hafta 7'de miras bağlamında, `public` sözleşme kavramı Hafta 10'da arayüzlerle, setter'daki sessiz ret yerine istisna fırlatma ise Hafta 11'de karşımıza çıkacaktır.

9. Haftanın Özeti

- **Kapsülleme iki bileşenlidir:** veriyi gizlemek ve kuralı sınıfın içine taşımak. İkincisi olmadan birincisi yalnızca uzun yazılmış bir `public` alandır.
- **Dört erişim düzeyi:** `private` · varsayılan (paket) · `protected` · `public`. Varsayılan düzey hiçbir şey yazmayarak elde edilir ve `protected`'tan **dardır**.
- **Alanlar `private` olur;** istisnası `public static final` sabitlerdir.
- **Getter ile setter zorunlu bir çift değildir;** yalnızca getter yazmak salt-okunur alan üretir, hiç kapı açmamak da geçerlidir.
- **Doğrulama setter'ın içindedir;** geçersiz değerde nesne eski durumunu korur. Yapıcı da doğrulamayı setter'a devreder.
- **`final` bağı dondurur, içeriği değil.** İçerik için değişmez tasarım gerekir: `final class`, `private final` alanlar, setter yok, dönüşümler yeni nesne döndürür.
- **Türetilebilen bilgi alanda tutulmaz;** Fahrenheit Celsius'tan hesaplanır. Böylece tutarsızlık imkânsız hâle gelir.
- **Her değişiklik bir alan ataması değildir;** bakiyeyi `setBakiye()` değil `yatir()` ve `cek()` değiştirir.

→ BİR SONRAKİ HAFTA

Hafta 5 — Yapıcı ve Yıkıcı Yöntemler. Aşırı yüklenmiş yapıcılar, `this(...)` zinciri, kopya yapıcı, statik fabrika metodu ve Java'da yıkıcı olmamasının nedeni.

10. Alıştırmalar

Alıştırmalar üç kümeye ayrılmıştır: kavram soruları (1–4), kod okuma ve hata bulma (5–7), programlama (8–12). Zorluk göstergesi: ●○○ temel, ●●○ orta, ●●● ileri. Derse gelmeden önce `Ogrenci` ve `ParaBirimi` sınıflarını gözden geçirmeniz önerilir.

10.1. Kavram Soruları

Alıştırma 1 ●○○

Kapsüllemenin iki bileşenini kendi cümlelerinizle yazınız. Yalnızca birinci bileşeni uygulayan bir sınıfın neden kapsüllenmiş sayılmadığını bir örnekle açıklayınız.

Alıştırma 2 ●●○

Aşağıdaki dört durumun her biri için hangi erişim düzeyinin uygun olduğunu gerekçesiyle yazınız: (a) bir sınıfın iç hesaplama yapan yardımcı metodu, (b) aynı pakette bulunan iki sınıfın paylaştığı bir bayrak alanı, (c) alt sınıfların kullanması beklenen bir yapı taşı metodu, (d) sınıfın dışa vaat ettiği ana işlem.

Alıştırma 3 ●●●

`protected` bir alanın, başka paketteki alt sınıfta neden `this.alan` ile erişilebilip `baskaNesne.alan` ile erişilemediğini açıklayınız. Bölüm 4.2'deki çıktıyı dayanak olarak kullanınız.

Alıştırma 4 ●●○

`final` bir alan ile değişmez bir sınıf arasındaki farkı açıklayınız. `private final int[] notlar;` bildirimi hangi güvenceyi verir, hangisini vermez?

10.2. Kod Okuma ve Hata Bulma

Alıştırma 5 ●●○

Aşağıdaki sınıfta kapsülleme açısından **üç ayrı** kusur vardır. Üçünü de bulup nasıl düzelteceğinizi yazınız.

```
1  public class Sepet {
2      public double toplam;              // (1)
3      private int adet;
4      private double birimFiyat;
```

```
5
6       public void setAdet(int a) { this.adet = a; }              // (2)
7       public void setBirimFiyat(double f) { this.birimFiyat = f; }
8       public double getToplam() { return toplam; }               // (3)
9   }
```

Kod 4.18 — Alıştırma 5 için inceleme kodu

Alıştırma 6 ●●○

Aşağıdaki kod parçası derlenir mi? Derlenirse çıktısı ne olur, derlenmezse hangi satırda hangi hatayı verir? Yanıtınızı çalıştırmadan önce yazınız, sonra doğrulayınız.

```
1   public class Kutu {
2       private final int[] degerler = {1, 2, 3};
3       public int[] getDegerler() { return degerler; }
4
5       public static void main(String[] args) {
6           Kutu k = new Kutu();
7           int[] d = k.getDegerler();
8           d[0] = 99;
9           System.out.println(k.getDegerler()[0]);
10      }
11  }
```

Kod 4.19 — Alıştırma 6 için inceleme kodu

Alıştırma 7 ●●○

`Sicaklik` sınıfına `private double fahrenheit;` diye ikinci bir alan eklendiğini varsayınız. `setCelsius()` metodunda bu alanı güncellemeyi unutan bir programcının yol açacağı tutarsızlığı somut bir çağrı dizisiyle gösteriniz.

10.3. Programlama Alıştırmaları

Alıştırma 8 ●○○

`Kitap` adında bir sınıf yazınız. Alanlar: `isbn` (salt-okunur), `baslik`, `sayfaSayisi`, `fiyat`. Gereksinimler: `sayfaSayisi` pozitif olmalı, `fiyat` negatif olmamalı, `baslik` boş olmamalı. Geçersiz istekler reddedilmeli ve nesne eski durumunu korumalıdır. `toString()` ezilmelidir.

İpucu: `Ogrenci` sınıfındaki setter düzenini örnek alınız; yapıcıda setter'ları çağırmayı unutmayınız.

Alıştırma 9 ●●○

`Sifre` adında bir sınıf yazınız. Şifre `private` tutulmalı ve **hiçbir getter ile dışarı verilmemelidir**. Sınıf yalnızca `boolean dogrula(String deneme)` ve `boolean degistir(String eski, String yeni)` metotlarını sunmalıdır. `degistir()` yalnızca eski şifre doğruysa ve yeni şifre en az 8 karakterse başarılı olmalıdır.

İpucu: Bu, "hiç getter yazmamak" tasarımının klasik örneğidir. `toString()` metodunuz da şifreyi yazdırmamalıdır.

Alıştırma 10 ●●○

`Zaman` adında **değişmez** bir sınıf yazınız. Alanlar: `saat` (0–23) ve `dakika` (0–59). Bölüm 4.4'teki beş kuralı uygulayınız. Sınıf `Zaman ekle(int dakika)` metodunu sunmalı ve **yeni bir nesne** döndürmelidir. Gün taşması başa dönmelidir (23:50 + 20 dk = 00:10).

İpucu: `ParaBirimi` ve `Koordinat` sınıflarını örnek alınız. Geçersiz değerler için yapıcıda `IllegalArgumentException` fırlatabilirsiniz.

Alıştırma 11 ●●●

İki paketten oluşan küçük bir düzen kurunuz: `depo` paketinde `Urun` ve `Ambar` sınıfları, `magaza` paketinde `Kasa` sınıfı. `Urun.stok` alanı **paket-özel** olsun; `Ambar` ona doğrudan erişebilsin ama `Kasa` erişemesin. `Kasa`, stok bilgisine yalnızca `Ambar`'ın `public` metotlarından ulaşabilsin.

İpucu: Föydeki Görev 5 (`kutuphane` paketi) tam olarak bu deseni kullanır; oradaki yapıyı örnek alabilirsiniz.

Alıştırma 12 ●●●

`Ogrenci` sınıfını genişletiniz: en fazla 10 not saklayabilen `private double[] notlar` alanı ekleyiniz. `notEkle()` diziyi doldursun; `getNotlar()` metodu diziyi **kopyalayarak** döndürsün. Ardından kopyalamayan bir sürüm yazıp iki sürümün farkını gösteren bir sürücü program yazınız.

İpucu: `dizi.clone()` sığ bir kopya üretir; ilkel tür dizileri için bu yeterlidir. Bölüm 6.4'ü yeniden okuyunuz.

11. Öz Değerlendirme

Aşağıdaki maddelerin tamamını işaretleyebiliyorsanız bu haftanın kazanımlarına ulaşmışsınız demektir. İşaretleyemediğiniz her madde için karşısındaki bölüme dönünüz.

Öz Değerlendirme Kontrol Listesi

- ☐ Kapsüllemenin iki bileşenini sayabiliyorum ve yalnızca birincisini uygulayan kodu tanıyabiliyorum. *(Bölüm 2, 3)*
- ☐ Dört erişim düzeyini görünürlük matrisiyle birlikte hatırlıyorum. *(Bölüm 4.1)*
- ☐ Varsayılan (paket) erişimin `protected`'tan **dar** olduğunu biliyorum. *(Bölüm 4.1, 4.2)*
- ☐ `protected` bir üyenin başka paketteki alt sınıfta neden yalnızca `this` üzerinden görüldüğünü açıklayabiliyorum. *(Bölüm 4.2)*
- ☐ Bir alan için getter/setter yazılıp yazılmayacağına gerekçeli karar verebiliyorum. *(Bölüm 4.5, Şekil 4.8)*
- ☐ Doğrulamalı setter yazabiliyor ve geçersiz değerde nesnenin eski durumunu koruduğunu gösterebiliyorum. *(Bölüm 3.3)*
- ☐ `final` alanın neyi dondurup neyi dondurmadığını ayırt edebiliyorum. *(Bölüm 4.3)*
- ☐ Değişmez sınıfın beş kuralını sayabiliyor ve uygulayabiliyorum. *(Bölüm 4.4)*
- ☐ Türetilmiş getter yazabiliyor, yedekli alan tutmanın sakıncasını açıklayabiliyorum. *(Bölüm 4.6)*
- ☐ Setter yerine anlamlı iş metodu yazmanın ne zaman doğru olduğunu biliyorum. *(Bölüm 4.5)*

12. Kaynaklar

12.1. Ders Kitapları

Kaynak	İlgili bölüm	Not
Deitel & Deitel, *Java How to Program* (11. baskı)	Bölüm 8.2–8.6 — Controlling Access to Members, `this`, `final`	Temel düzey, bol örnekli
Horstmann, *Core Java Volume I* (12. baskı)	Bölüm 4.3–4.6 — Encapsulation, Access Modifiers, Packages	Paket kavramı için en açıklayıcı anlatım
Schildt, *Java: The Complete Reference* (12. baskı)	Bölüm 6 — Introducing Classes; Bölüm 9 — Packages and Interfaces	Başvuru kaynağı
Bloch, *Effective Java* (3. baskı)	Madde 15 — Minimize the accessibility of classes and members; Madde 16 — In public classes, use accessor methods, not public fields; Madde 17 — Minimize mutability	Bu haftanın tasarım felsefesinin kaynağı

Tablo 4.12 — Bu hafta için önerilen okumalar

12.2. Çevrimiçi Kaynaklar

- **Oracle Java Tutorials — Controlling Access to Members of a Class:** erişim düzeylerinin resmî tablosu ve örnekleri.
- **Oracle Java Tutorials — Creating and Using Packages:** paket bildirimi, klasör düzeni ve `import` kullanımı.
- **Java Language Specification, §6.6 — Access Control:** erişim kurallarının biçimsel tanımı (ileri düzey okuma).
- **Oracle Java Tutorials — A Strategy for Defining Immutable Objects:** değişmez sınıf tasarımının resmî kılavuzu.

12.3. İletişim ve Destek

Konu	Sorumlu	Kanal
Teori, kavram soruları	Dr. Öğr. Üyesi Rıfat BENVENİSTE	Ders saati sonrası · e-posta
Laboratuvar, kod hataları	Arş. Gör. Betül MUMCU	Laboratuvar saatleri · e-posta
Teslim ve puanlama	Arş. Gör. Betül MUMCU	Ders yönetim sistemi
Ortam kurulumu (JDK, VS Code)	Laboratuvar sorumlusu	Hafta 1 föyü, Görev 0

Tablo 4.13 — Soru ve destek için başvuru noktaları

> **ⓘ BİLGİ**
>
> Bu haftanın tüm Java kaynak dosyaları `Hafta_04/Java_Kodlari/` klasöründedir: ders örnekleri `ders/`, laboratuvar çözümleri ise laboratuvar oturumundan sonra `laboratuvar/` altında paylaşılacaktır. Kod bloklarındaki satır numaraları bu dosyalarla birebir örtüşür.